4-bit Array Multiplier from RTL to GDSII using Open-Source Tools

Digital VLSI Design (M26EC2.408), Assignment Report

Vansh Jaiswal (2024102013) Saarthak Sabharwal (2023102055)

Repository: https://github.com/VaNsH-IIIT/DVD-A1-array-multiplier

1 Introduction

In this assignment we built a 4-bit × 4-bit unsigned array multiplier in Verilog and took it all the way from RTL to a final layout on the SkyWater SKY130 (130 nm) process, using the `sky130_fd_sc_hd` standard cell library. The flow had four parts:

- Part A: writing the RTL and checking it with an exhaustive testbench (Icarus Verilog, GTKWave),
- Part B: synthesis to SKY130 cells with Yosys, and simulating the netlist,
- Part C: static timing analysis with OpenSTA,
- Part D: place and route with LibreLane (OpenROAD, Magic, KLayout, Netgen) to get the GDSII.

We ran all the tools from the IIC-OSIC-TOOLS Docker image (`hpretl/iic-osic-tools:latest`) with `PDK=sky130A`, except GTKWave, which we ran on the host to view the waveform. The versions were Icarus Verilog 14.0, Yosys 0.69, OpenSTA 3.1.0, LibreLane 3.1.0.dev3, Magic 8.3, Netgen 1.5 and KLayout 0.30. All the commands we ran, in order, are in the `README.md` of our repository.

2 Part A: RTL Design and Verification

2.1 Design

Full adder. `rtl/full_adder.v` has two `assign` statements: $sum = a \oplus b \oplus c_{in}$ and $c_{out} = ab + a\,c_{in} + b\,c_{in}$.

Multiplier. `rtl/mult_array.v` has the ports `clk`, `rst_n`, `a[3:0]`, `b[3:0]` and `p[7:0]`. On every rising edge it stores `a` and `b` in two 4-bit registers `a_r` and `b_r`. The 16 partial product bits $pp_i[j] = a_r[j] \cdot b_r[i]$ come from 16 AND gates. Row 0 has no adders: its bit 0 is directly $p[0]$ and the other three bits go down to row 1. Rows 1, 2 and 3 are each a 4-bit ripple-carry adder made of 4 `full_adder` instances (`fa1_0` to `fa3_3`, 12 in total), connected as given in the table in Section 2 of the assignment. In each row, the lowest sum bit is a final product bit, and the other three sum bits plus the last carry go to the next row. Row 3 also gives $p[7:4]$. The 8-bit result is stored in the output register `p`.

So the design has $4 + 4 + 8 = 16$ flip-flops, all reset to 0 by the active-low asynchronous reset `rst_n`. A new input is captured at clock edge 1 and its product reaches `p` at edge 2, so the latency is 2 cycles, and one new result comes out every cycle. We did not use the `*` operator anywhere in the design. Figure 1 shows the block diagram.

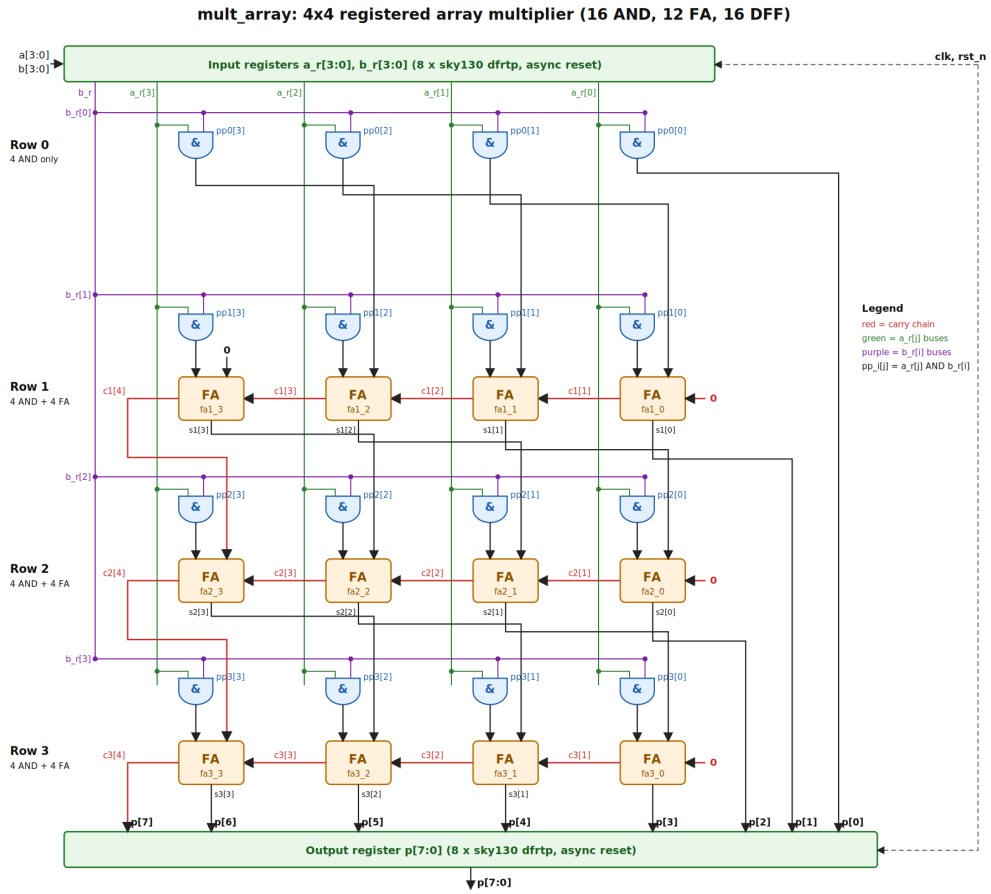


Figure 1: Block diagram of `mult_array`: input registers (8 flip-flops), the 16 AND gates, the 12 full adders in three rows (carry chains in red) and the 8-bit output register.

2.2 Testbench

Our testbench `tb/tb_mult.v` makes a 10 ns clock and keeps `rst_n` low for the first 2 cycles. Then it goes through all 256 pairs $(a, b = 0 \ldots 15)$. Each pair is applied on a falling edge. The testbench then waits for two rising edges (the latency) and compares `p` with `a*b`, 4 ns after the second edge. The `*` operator is only used here, in the testbench. Every mismatch is printed, and at the end it prints one summary line. It also dumps the waveform to `mult.vcd`.

We first sampled `p` 1 ns after the edge. That worked for the RTL, but in the gate-level simulation (Part B) every result came out one vector late, because the flip-flop models with `UNIT_DELAY` also change their output 1 ns after the edge. Moving the check to 4 ns fixed this. The design was not changed.

2.3 Results

The RTL simulation printed:

```
PASS: 256 vectors checked, 0 errors
```

To check that the testbench really catches bugs, we changed `sum` in `full_adder.v` to `a ^ b` for a moment and ran it again:

```
MISMATCH at t=1069 ns: a=3 b=3 p=5 expected=9
MISMATCH at t=1129 ns: a=3 b=6 p=10 expected=18
...
FAIL: 256 vectors checked, 95 errors
```

After that we restored the line and got PASS again. The first failure, 3×3, is the first case in which a carry has to enter a full adder: the broken adder drops it, giving 5 instead of 9. Only 95 of the 256 cases fail, because the bug only matters when a full adder actually gets a carry-in of 1. Products like $x \times 0$, $x \times 1$ or $x \times 2$ never make an internal carry, so they still come out right. This is why the test has to go through all 256 cases. Figure 2 shows both runs.

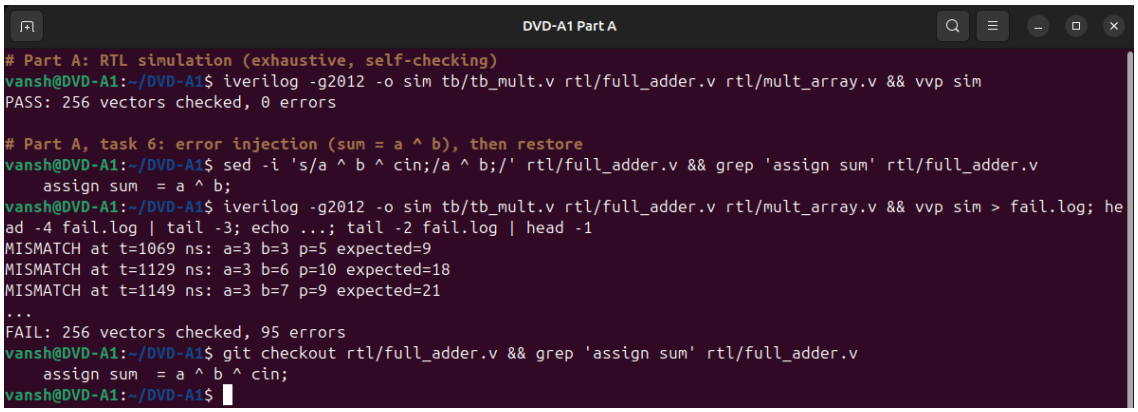


Figure 2: Terminal output of Part A: PASS with the correct full adder, FAIL (95 errors) with `sum = a ^ b`, then the file restored.

2.4 Hand-worked example

We used Vansh's roll number, 2024102013. The last digit is 3, so $A = 3+5 = 8$ and $B = 7$ (A is not 7, so B stays 7). Figure 3 shows the multiplication done by hand, $1000 \times 0111 = 0011\,1000 = 56$.

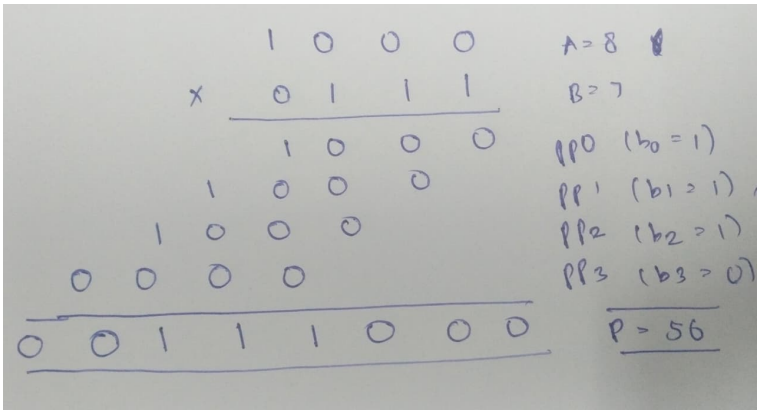


Figure 3: Hand-worked example: $8 \times 7 = 56$, with the four shifted partial products $pp_0 \ldots pp_3$.

We also followed the same numbers through the rows of our array (Table 1). The values match the internal signals we saw in the simulation.

Table 1: 8×7 going through the array.

Row	What is added	Sum / carry out	Product bit	Passed down
0	$pp_0 = 1000$	–	$p[0] = 0$	$r_0 = 0100$
1	$1000 + 0100$	1100 / 0	$p[1] = 0$	$r_1 = 0110$
2	$1000 + 0110$	1110 / 0	$p[2] = 0$	$r_2 = 0111$
3	$0000 + 0111$	0111 / 0	$p[3] = 1$	$p[7:4] = 0011$

So $P = 0011\,1000_2 = 56 = 8 \times 7$.

2.5 Waveform

Figure 4 shows this example in GTKWave. `a = 8` and `b = 7` are applied at 2730 ns, they go into `a_r` and `b_r` at the next rising edge (2735 ns, edge 1), and `p = 56` shows up at 2745 ns (edge 2). That is 20 ns, i.e. 2 clock cycles. Until edge 2, `p` still holds 48, the product of the previous pair (8×6).

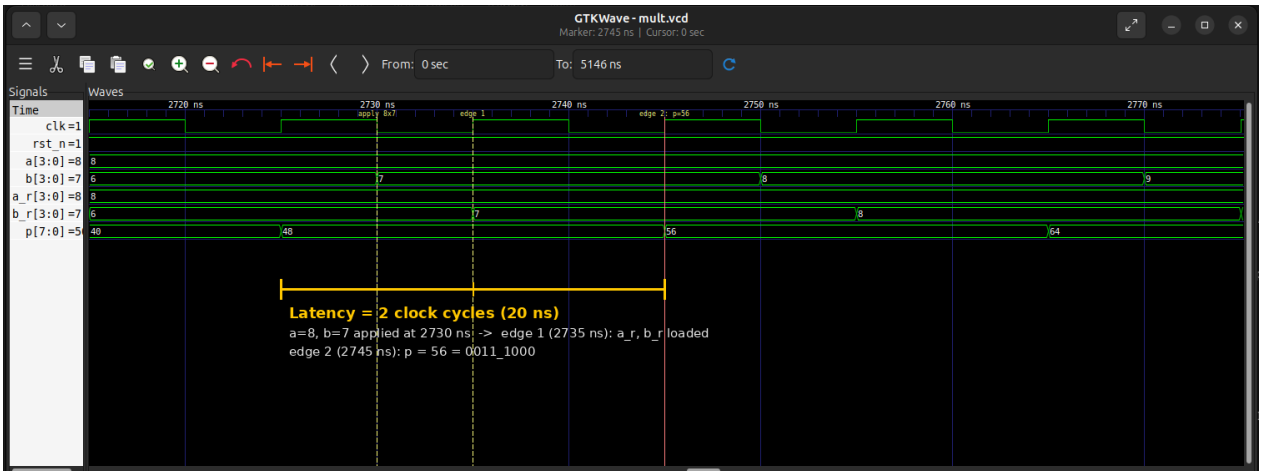


Figure 4: GTKWave waveform for 8×7, with the 2-cycle latency marked: inputs at 2730 ns, input registers at edge 1 (2735 ns), `p = 56` at edge 2 (2745 ns).

3 Part B: Logic Synthesis with Yosys

Our `synth/synth.tcl` runs the seven steps from the assignment: `read_verilog`, `synth -top mult_array -flatten`, `dfflibmap` and `abc` with the typical corner library `sky130_fd_sc_hd__tt_025C_1v80.lib`, `opt_clean -purge`, `stat -liberty` (saved as `synth/mult_array_stat.rpt`) and `write_verilog -noattr` (saved as `synth/mult_array_netlist.v`). We ran it with `yosys -c synth/synth.tcl`. Table 2 has the results.

Table 2: Part B results (from `synth/mult_array_stat.rpt`).

Metric	Value
Total number of cells	63
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area	740.710 μm^2
Flip-flop area	$16 \times 25.024 = 400.384$ μm^2
Flip-flop area (% of total)	54.05 %
3 most-used cell types	`dfrtp_1` $\times$ 16, `nand2_1` $\times$ 8, `a21oi_1` $\times$ 6
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

The remaining 33 cells are `xnor2_1` $\times$ 4, `maj3_1` $\times$ 3, `nand3_1` $\times$ 3, `xor2_1` $\times$ 3, `and2_0` $\times$ 2, `clkinv_1` $\times$ 2, `lpflow_isobufsrc_1` $\times$ 2, `xnor3_1` $\times$ 2, and 12 other cell types used once each.

3.1 Gate-level simulation

To make sure synthesis did not change what the circuit does, we simulated the netlist with the same, unchanged testbench, together with the SKY130 cell models:

```
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls tb/tb_mult.v \
  $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
vvp gls
PASS: 256 vectors checked, 0 errors
```

The netlist only has the 63 SKY130 cells, with no RTL left in it, so this run really tests the synthesized gates. Table 3 compares the two simulations. Since all 256 inputs pass, the netlist behaves the same as the RTL, with the same 2-cycle latency and reset behaviour (Figure 5 shows this run).

Table 3: RTL simulation versus gate-level simulation, same testbench.

	RTL (Part A)	Gate-level netlist (Part B)
Design simulated	`full_adder.v` + `mult_array.v`	`mult_array_netlist.v` (63 cells)
Testbench	`tb/tb_mult.v`	`tb/tb_mult.v` (unchanged)
Vectors checked	256	256
Errors	0	0
Simulation end time	5149 ns	5149 ns
Result	PASS	PASS

4 Part C: Static Timing Analysis with OpenSTA

`sta/sta.tcl` reads the typical corner library and our netlist, creates a clock of period T on `clk`, puts a 1.0 ns input delay on every input except `clk` and a 1.0 ns output delay on the outputs, prints the worst setup slack, and writes the critical path to `sta/mult_array_timing.rpt`. The period comes from the variable `T` (default 2.60 ns).

First we ran it with $T = 10$ ns (`T=10 sta sta/sta.tcl`). The worst slack was 7.396 ns, so

$$T_{min} = 10 - 7.396 = 2.604 \text{ ns}, \qquad F_{max} = \frac{1000}{2.604} = 384.0 \text{ MHz}.$$

Then we ran it again with $T = 2.60$ ns (`sta sta/sta.tcl`) and got a slack of -0.004 ns, which is effectively 0, as expected. (At 2.61 ns the slack is $+0.006$ ns.) The critical path is the same in both runs, because with an ideal clock the path delay does not depend on T. The committed timing report is from this second run.

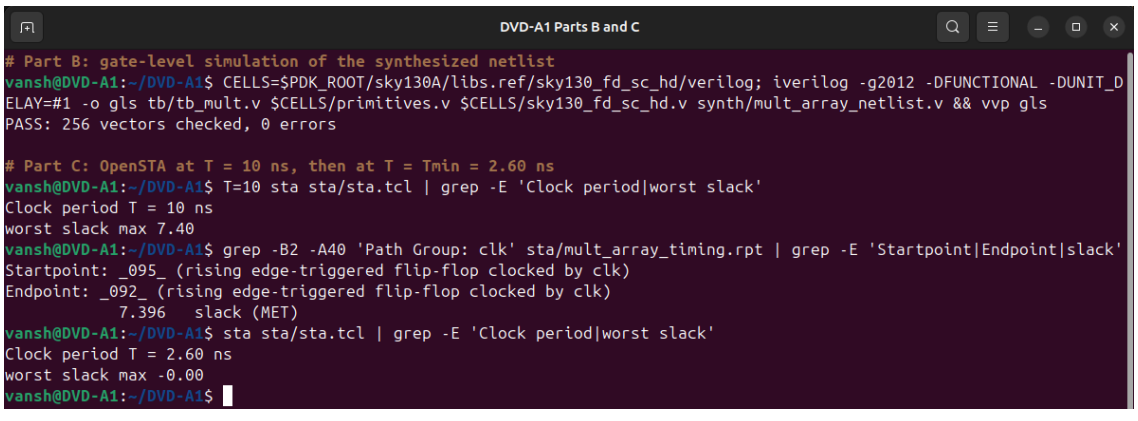


Figure 5: Terminal output of Parts B and C: gate-level simulation PASS, then OpenSTA with worst slack 7.40 ns at $T = 10$ ns and -0.00 ns at $T = 2.60$ ns.

Table 4: Part C results (typical corner, `tt_025C_1v80`).

Metric	Value
Worst slack at $T = 10\,\text{ns}$	7.396 ns
T_{min}	$10 - 7.396 = 2.604\,\text{ns}$ ($\approx 2.60\,\text{ns}$)
F_{max}	384.0 MHz
Slack at $T = T_{min}$ (2.60 ns)	$-0.004\,\text{ns}$ (≈ 0)
Critical path start point	input register `a_r[1]` (instance `_095_`)
Critical path end point	output register `p[6]` (instance `_092_`)
Number of cells on the critical path	9 (launch flip-flop + 8 logic cells)
Latency $= 2 \times T_{min}$	5.208 ns
Throughput	384.0 million results per second

4.1 Cells on the critical path

Table 5 lists the cells on the path in order. The netlist names do not say which full adder a gate belongs to, so we worked it out ourselves: for each net we compared its value over all 256 input pairs with the signals of our RTL.

Table 5: Critical path, in order (from `sta/mult_array_timing.rpt`).

#	Cell	Delay (ns)	Time (ns)	Signal	Belongs to
0	`dfrtp_1`	0.398	0.398	`a_r[1]`	input register
1	`clkinv_1`	0.074	0.472	inverted `a_r[1]`	AND gates
2	`o311ai_0`	0.348	0.820	(merged)	AND gates + fa1_1
3	`a21oi_1`	0.172	0.992	$c_2[1]$	fa2_0 carry
4	`maj3_1`	0.381	1.373	$c_2[2]$	fa2_1 carry
5	`xnor2_1`	0.160	1.534	inverted $s_2[2]$	fa2_2 sum
6	`maj3_1`	0.385	1.918	inverted $c_3[2]$	fa3_1 carry
7	`maj3_1`	0.377	2.295	inverted $c_3[3]$	fa3_2 carry
8	`xnor3_1`	0.175	2.471	$s_3[3]$	fa3_3 sum
–	`dfrtp_1`	–	–	`p[6]`	output register (setup 0.134 ns)

In total: 0.398 (clock to Q) + 2.073 (logic) + 0.134 (setup) = 2.605 ns, which is our T_{min}. The path goes through **7 full-adder stages**: fa1_1 → fa2_0 → fa2_1 → fa2_2 → fa3_1 → fa3_2 → fa3_3. It runs along the carry chain of row 2, goes down through $s_2[2]$, and then along the carry chain of row 3 to the top sum bit. This is close to the estimate of about $2N = 8$ full-adder delays. Six of the eight logic cells are directly a carry or a sum of one full adder, and each stage costs only one gate (about 0.30 ns), because the path only needs the carry or the sum of each adder, not both.

5 Part D: Physical Design with LibreLane

`pnr/config.json` has `DESIGN_NAME = "mult_array"`, the two RTL files, `CLOCK_PORT = "clk"`, `FP_SIZING = "absolute"` and `DIE_AREA = [0, 0, 150, 150]`. The first clock period was $1.2 \times T_{min} = 1.2 \times 2.604 = 3.125\,\text{ns}$, rounded up to 3.5 ns. We ran the flow with `librelane config.json` inside `pnr/`, and every time timing failed we added 0.5 ns and ran it again. Table 6 lists all the runs.

Table 6: All LibreLane runs (worst slack = worst of the 9 signoff corners).

Run	Period (ns)	Worst slack (ns)	Typical slack (ns)	DRC	LVS	Result
1	3.5	−2.132	+0.547	0	0	fail
2	4.0	−1.632	+1.047	0	0	fail
3	4.5	−1.142	+1.551	0	0	fail
4	5.0	−0.642	+2.052	0	0	fail
5	5.5	−0.142	+2.552	0	0	fail
6	6.0	−0.064	+2.943	0	0	fail
7	6.5	+0.085	+3.293	0	0	**pass**

DRC and LVS were clean in every run. Only setup timing failed, and only in the slow corner `max_ss_100C_1v60` (slow transistors, 100 °C, 1.60 V), where the cells are roughly twice as slow as in the typical corner. The typical corner, which is the one we used in Part C, passed from the very first run. Runs 1 to 5 got better by almost exactly 0.5 ns each time, but run 6 only gained 0.08 ns. This is because LibreLane synthesizes the design again for every clock period, and with a looser clock it picks smaller and slower gates.

Table 7: Part D results (final run, from `pnr/metrics.json`).

Metric	Value
CLOCK_PERIOD used	6.5 ns
Number of runs needed	7
Core area	17 759.5 μm^2 (die: $150 \times 150 = 22\,500\,\mu$m^2)
Cells after synthesis (Part B)	63
Cells after place and route	1 901
Kinds of cells added	decap, fill, tap, I/O buffers, clock buffers, fanout buffers (Table 8)
Wirelength	2 405 μm
Setup slack	+0.085 ns (worst corner)
Post-route F_{max}	$1000/(6.5 - 0.085) = 155.9$ MHz
F_{max} from Part C	384.0 MHz
DRC errors	0 (Magic 0, KLayout 0, router 0)
LVS errors	0 (Netgen)

Other results from the final run: hold slack +0.124 ns, no antenna violations, no power grid violations, and the cells use 7.0 % of the core. There were 11 max-slew warnings, all in the slow corner and all on the reset net (0.761 ns against a 0.75 ns limit). The reset is asynchronous, the typical corner has none of these warnings, and the flow finished normally.

Table 8: The 1 901 cells after place and route (from the final DEF).

Kind	Cells	Count
Tap cells	`tapvpwrvgnd_1`	245
Decap cells	`decap_3`, `decap_4`, `decap_6`, `decap_8`, `decap_12`	1 296
Fill cells	`fill_1`, `fill_2`	270
Clock tree buffers	`clkbuf_16`	3
I/O buffers	`buf_2` (9 inputs, 8 outputs)	17
Fanout buffers	`clkdlybuf4s50_1`	2
Design cells	`dfrtp_1` × 16, 49 logic gates, `inv_1` × 3	68
Total		1 901

LibreLane runs its own synthesis, which gave 68 design cells instead of the 63 from Part B.

5.1 Layout

Figure 6 shows the whole die in KLayout and Figure 7 shows a zoomed view of the logic, where the standard cell rows and the metal wires can be seen.

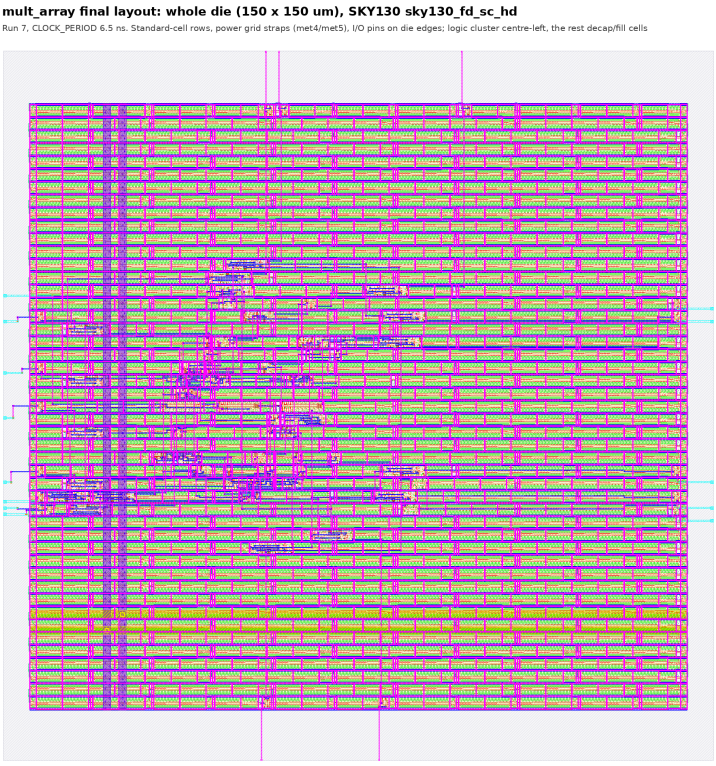


Figure 6: Final layout, the whole $150 \times 150\,\mu$m die. The I/O pins are on the edges and the power straps run over the cell rows. The logic is on the centre-left, and the rest of the rows are filled with decap and fill cells.

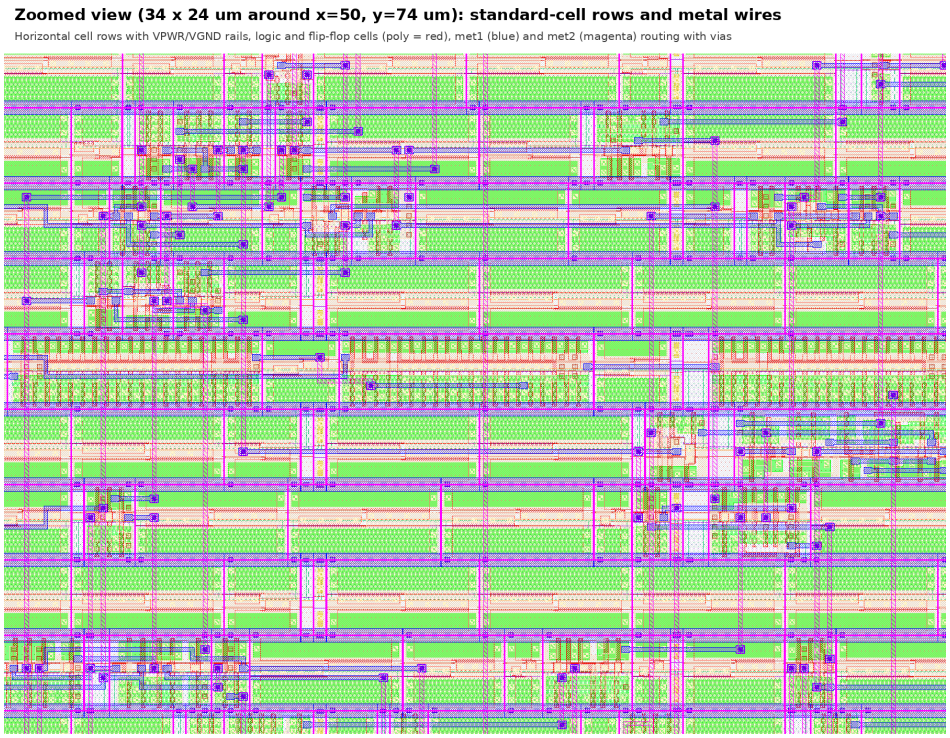


Figure 7: Zoomed view ($34 \times 24\,\mu$m) of the logic: standard cell rows between the VPWR/VGND rails, the cells (polysilicon in red), and metal 1 (blue) and metal 2 (magenta) wires with vias.

6 Answers to the Questions

Q1. Logic cells after synthesis

Synthesis produced $63 - 16 =$ **47 logic cells** ($340.3\,\mu\text{m}^2$), not $16 + 12 = 28$. The 16 AND gates and 12 full adders describe our circuit, but after `-flatten` ABC maps the whole logic onto the library cells that fit it best, so the count changes in both directions:

- **More cells:** ABC does not use the full adder cell (`fa_1`), so each full adder becomes a separate sum cell (`xor`/`xnor`, sometimes the 3-input versions) and a carry cell (`maj3`, or AOI/OAI/NAND gates). Two inverters (`clkinv_1`) are also added to fix signal polarities.
- **Fewer cells:** most AND gates are merged into larger cells such as `o311ai`, `a21oi` and `nand2`; the first adder of each row has $c_{in} = 0$ and reduces to a half adder; and the constant 0 fed into row 1 removes more logic.

The netlist still has exactly the same function as the RTL, as the gate-level simulation showed.

Q2. Critical path

The critical path starts at the input register `a_r[1]` and ends at the output register `p[6]`. It has **9 cells**: the launch flip-flop and 8 logic cells (`clkinv`, `o311ai`, `a21oi`, `maj3`, `xnor2`, `maj3`, `maj3`, `xnor3`), with a data arrival time of 2.471 ns and a setup time of 0.134 ns (Table 5).

Matching each net to our RTL shows that the path crosses **7 full-adder stages**: along the row 2 carry chain, down through $s_2[2]$, and along the row 3 carry chain to $s_3[3]$. This is close to the estimate of about $2N = 8$ full-adder delays, which is only a rough count of the longest chain. Each stage costs a single gate (about 0.30 ns), because the path needs either the carry or the sum of each adder, never both.

Q3. Flip-flop area

The 16 flip-flops take 400.384/740.710 = **54.05** % of the cell area, although they are only 16 of the 63 cells. Two reasons:

- **A flip-flop is a large cell.** A `dfrtp_1` is a master–slave flip-flop with asynchronous reset, built from 28 transistors (a `nand2_1` has 4). At 25.02 μm^2 it is the largest cell in our design: about 6.7 times a `nand2_1` (3.75 μm^2) and larger than any logic cell we use (the largest, `xor3_1`, is 23.77 μm^2).
- **The design is very small.** The array has only 28 AND/full-adder blocks, while registering both inputs and the output already needs $4N = 16$ flip-flops.

Registers grow with N but the array grows with N^2. For $N = 8$ we would expect 32 flip-flops (800.8 μm^2) against about 120 blocks ($\approx$ 1460 μm^2 at our 12.15 μm^2 per block), so the flip-flop share would fall to about 35 %.

Q4. Post-route F_{max} and added cells

After routing, F_{max} = **155.9 MHz**, compared with **384.0 MHz** in Part C. Part C was an idealised analysis; four effects make the routed design slower:

1. **Process corner.** Part C used only the typical corner, but LibreLane signs off in the slow corner (100 °C, 1.60 V), where cells are about twice as slow. With the typical-corner slack of the routed design (+3.293 ns) we get $1000/(6.5 - 3.293) = 311.8$ MHz, so most of the gap comes from the corner.
2. **Wires.** Routed nets have resistance and capacitance, so gates drive larger loads and switch more slowly. Part C had no wires.
3. **Clock tree.** The clock is no longer ideal: it passes through two levels of `clkbuf_16` (about 0.5 ns, with some skew between flip-flops), and the flow keeps 0.25 ns of clock uncertainty.
4. **Re-synthesis.** LibreLane synthesizes the design again for the target period and uses smaller, slower gates when the clock is loose.

The flow added 1 833 cells (68 → 1 901). Table 9 lists each kind and why it is needed.

Table 9: Cells added by the flow and why each kind is needed.

Kind	Count	Why it is needed
Tap cells	245	Connect the n-well to VPWR and the substrate to VGND along every row, to prevent latch-up.
Decap cells	1 296	Small capacitors between VPWR and VGND that supply charge when gates switch, reducing supply noise and IR drop.
Fill cells	270	Fill the leftover gaps so that wells, implants and power rails are continuous along each row, as DRC requires.
Clock buffers	3	The clock tree (`clkbuf_16`) that delivers `clk` to all 16 flip-flops with low skew.
I/O buffers	17	One `buf_2` on each of the 9 input and 8 output pins, separating the pins from the internal logic.
Fanout buffers	2	Split the reset net, which drives all 16 flip-flops.

Q5. DRC and LVS

DRC (design rule check) verifies that the shapes in the layout follow the foundry's manufacturing rules: minimum width and spacing on every layer, enclosure of contacts and vias, overlaps, density and antenna rules. **LVS** (layout versus schematic) extracts the transistors and their connections from the layout and checks that they form exactly the same circuit as the gate-level netlist, with the same devices, nets and pins.

Both must pass because they catch different problems. A DRC error means the chip may not be manufacturable, or may get shorts and opens that lower the yield. An LVS error means the chip could be manufactured perfectly and still be the wrong circuit. A layout can pass one check and fail the other, and errors found after fabrication are very expensive, so both must be clean before tape-out. Our final layout has **0 DRC errors** (Magic, KLayout and the router) and **0 LVS errors** (Netgen).

Q6. Scaling to an 8-bit multiplier

Using the formulas from Section 2 of the assignment with $N = 8$ instead of $N = 4$:

Table 10: 4-bit versus 8-bit array multiplier.

Quantity	$N = 4$	$N = 8$	Growth
Full adders, $N(N-1)$	12	56	4.7× (quadratic)
AND gates, N^2	16	64	4× (quadratic)
Flip-flops, $4N$	16	32	2× (linear)
Critical path, $\approx 2N$ FA delays	8	16	2× (linear)

So the hardware grows with N^2, but the delay only grows with N. Using our own Part C numbers (0.398 ns clock to Q, about 0.296 ns per full-adder stage, 0.134 ns setup):

$$T_{min,8} \approx 0.398 + 16 \times 0.296 + 0.134 = 5.27 \text{ ns}, \qquad F_{max,8} \approx \frac{1000}{5.27} \approx 190 \text{ MHz},$$

compared with 2.604 ns and 384.0 MHz for our 4-bit design (typical corner).

7 Team Contributions

We split the work equally. By the marks in the assignment, each of us did 50 of the 100: Parts A and C ($30 + 20$) and Parts B and D ($20 + 30$). Each of us also answered the questions that belong to our own parts.

Member	Work
Vansh Jaiswal (2024102013)	Part A: `full_adder.v`, `mult_array.v`, the testbench and its sampling fix, the error test, the hand-worked example and the waveform. Part C: `sta.tcl`, T_{min} and F_{max}, tracing the critical path through the full adders. Questions 2 and 6. The GitHub repository and the README.
Saarthak Sabharwal (2023102055)	Part B: `synth.tcl`, synthesis, gate-level simulation and the cell/area table. Part D: `config.json`, the LibreLane runs, the added-cell analysis, looking at the layout in Magic and KLayout, DRC/LVS signoff and the layout pictures. The block diagram. Questions 1, 3, 4 and 5.

We both wrote the report and checked each other's results. All commits were pushed from Vansh's GitHub account, so the commit history only shows one author.

8 Conclusion

Our 4-bit array multiplier passed all 256 test vectors in both the RTL and the gate-level simulation, and the testbench caught the injected full adder bug (95 errors). Synthesis gave 63 cells (740.71 μm^2, 54.05 % flip-flop area). Static timing in the typical corner gave $T_{min} = 2.604$ ns ($F_{max} = 384.0$ MHz), with the critical path going from `a_r[1]` through 7 full-adder stages to `p[6]`. LibreLane produced a DRC and LVS clean layout on a 150×150 μm die at a 6.5 ns clock (post-route $F_{max} = 155.9$ MHz). The big drop from 384.0 to 155.9 MHz comes mainly from signing off in the slow corner, and also from the wires, the real clock tree and the clock uncertainty, which the pre-layout analysis does not include.

Use of AI Tools

An AI assistant was used to support parts of this work. We guided it throughout, reviewed and checked every step and result ourselves, and we take full responsibility for the design and the report, in line with the responsible use of AI.

References

[1] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson, ch. 11.

[2] J. Rabaey, A. Chandrakasan and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Pearson.

[3] IIC-OSIC-TOOLS, https://github.com/iic-jku/IIC-OSIC-TOOLS

[4] SkyWater SKY130 PDK, https://skywater-pdk.readthedocs.io

[5] Icarus Verilog, https://steveicarus.github.io/iverilog

[6] Yosys, https://yosyshq.readthedocs.io/projects/yosys

[7] OpenSTA, https://github.com/parallaxsw/OpenSTA

[8] LibreLane, https://github.com/librelane/librelane

[9] OpenROAD, https://openroad.readthedocs.io

[10] Magic and Netgen, http://opencircuitdesign.com

[11] KLayout, https://www.klayout.de